Model Context Protocol Integration in Seismic Command: Architecture, Implementation, and Live Demo

Yunus Emre Balci
Akdeniz University — Computer Engineering

May 2026

Stack: FastMCP (Python) • Angular 18 • LangGraph • Spring Boot

Contents

1 Introduction **3**

2 System Architecture **3**
2.1 High-Level Overview . . . 3
2.2 Protocol Stack . . . 3

3 MCP Server Implementation **4**
3.1 Server Initialisation . . . 4
3.2 Registered Tools . . . 4
3.3 Example Tool: `assess_building_risk` . . . 5

4 MCP Client Implementation **5**
4.1 Streamable HTTP Transport . . . 5
4.2 Graph Workflow Integration: `call_mcp_tool` . . . 6
4.3 Demo Walkthrough: `run_mcp_demo` . . . 6

5 LangGraph Integration **7**
5.1 Hybrid Classifier and Tool Routing . . . 7
5.2 Category-to-Tool Mapping . . . 7
5.3 Error Handling . . . 7

6 Live Demo Page **8**
6.1 Architecture . . . 8
6.2 Demonstration Tools . . . 8
6.3 Protocol Steps Rendered in the UI . . . 8

7 Design Decisions **8**
7.1 Streamable HTTP vs. stdio Transport . . . 8
7.2 Single MCP Server for Multiple Consumers . . . 9
7.3 Deterministic Tool vs. LLM Node . . . 9

8 Test Scenarios **9**
8.1 Scenario 1: Real-Time Earthquake Query . . . 9
8.2 Scenario 2: Multi-Source Seismic Context . . . 9

8.3 Scenario 3: Building Risk Assessment 10
8.4 Scenario 4: AI Assistant — MCP in the Chat Flow 10

9 Reflection and Conclusion **10**

1 Introduction

Seismic Command is a full-stack earthquake intelligence platform that combines a Spring Boot data backend, a Python LangGraph orchestration service, and an Angular frontend. This report documents the integration of the **Model Context Protocol (MCP)** into Seismic Command and explains why MCP was chosen, how it was implemented, and how the protocol works end-to-end.

Design Rationale

MCP (Model Context Protocol) is an open protocol developed by Anthropic that gives AI models a *standardised interface* to tools, resources, and data sources. Without MCP, every consumer of the earthquake data (the chat agent, the risk assessment, a future mobile app, an external AI assistant) would need its own bespoke integration. With MCP, a single server exposes all capabilities and any compliant host can discover and invoke them automatically.

Concretely, Seismic Command benefits from MCP in three ways:

- **Standardisation:** tool schemas are discovered at runtime via `tools/list`; no hard-coded API surface in every consumer.
- **Reuse:** the LangGraph chat agent and the live demo page both talk to *the same* MCP endpoint.
- **Openness:** Claude Desktop, Cursor, or any MCP-capable host can be pointed at `http://localhost:8002/mcp/` and gain instant access to all earthquake tools without additional code.

Source code: https://github.com/YunusEmre3757/advenced_web.git

2 System Architecture

2.1 High-Level Overview

The system has four tiers. Figure 1 shows how they relate.

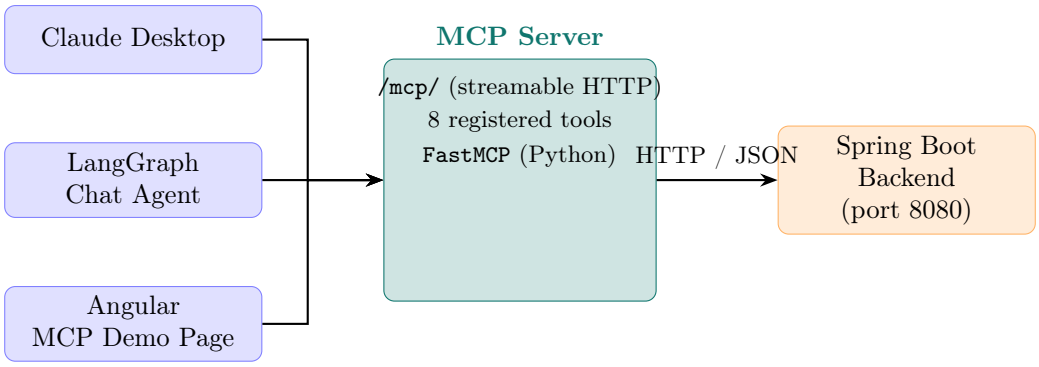

Figure 1: High-level MCP architecture. Three consumers share one MCP server.

2.2 Protocol Stack

The MCP server runs inside the same Python process as the LangGraph service. Table 1 lists every component involved.

Layer	Component	Technology
Data	Spring Boot backend	Java, REST/JSON
MCP Server	`seismic_server.py`	FastMCP, Python
MCP Client	`seismic_client.py`	mcp-python SDK, streamable HTTP
Orchestration	LangGraph chat graph	Python, LangChain
API Gateway	`api.py`	FastAPI, Uvicorn
Frontend	Angular 18 SPA	TypeScript, Signals

Table 1: Technology stack for the MCP integration.

3 MCP Server Implementation

3.1 Server Initialisation

The server is created with `FastMCP` and mounted onto the FastAPI application so that it is reachable at `/mcp/`:

```python
# seismic_server.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP(
    "seismic-command-mcp",
    instructions="Provides earthquake-domain tools for Seismic Command.",
    streamable_http_path="/",
)
```

```python
# api.py  mount + lifespan
seismic_mcp_app = seismic_mcp.streamable_http_app()

@asynccontextmanager
async def lifespan(_app):
    async with seismic_mcp.session_manager.run():  # MCP alive
        yield
    ...

app.mount("/mcp", seismic_mcp_app)   # reachable at /mcp/
```

Key Insight

Mounting the MCP server *inside* the FastAPI app means no separate process is required. Claude Desktop, Cursor, or any other MCP host simply adds `http://localhost:8002/mcp/` as an HTTP MCP server and instantly has access to all eight tools without any extra configuration.

3.2 Registered Tools

Table 2 documents all eight tools exposed by the server.

Tool	Key parameters	Spring endpoints used
`get_recent_earthquakes`	hours, min_magnitude, limit	`/api/earthquakes/recent`
`get_seismic_context`	latitude, longitude, radius_km	recent + historical + faultlines (parallel)
`get_earthquake_detail`	event_id	`/api/earthquakes/{id}` + aftershocks + similar + dyfi + shakemap
`get_historical_events`	years, min_magnitude	`/api/historical/events`
`get_seismic_gaps`	—	`/api/historical/gaps`
`get_fault_lines`	bbox (min/max lon/lat)	`/api/fault-lines`
`geocode_address`	query	`/api/geocode/search`
`assess_building_risk`	lat, lon, year, floors, soil, system	recent + historical + faultlines (parallel)

Table 2: MCP tools registered on the server.

3.3 Example Tool: `assess_building_risk`

This tool demonstrates the power of composing multiple data sources inside a single MCP call. It uses the same `get_seismic_context` tool internally for the seismic data, then applies a deterministic rule engine to produce a 0–100 risk score:

```
@mcp.tool()
async def assess_building_risk(
    latitude: float, longitude: float,
    construction_year: int, floor_count: int,
    soil_type: str = "ZC", structural_system: str = "RC",
    visible_damage: bool = False, radius_km: float = 100.0,
) -> dict[str, Any]:
    """Deterministic structural + seismic risk score (no LLM)."""
    context = await get_seismic_context(
        latitude=latitude, longitude=longitude, radius_km=radius_km
    )
    score = 0
    if construction_year < 1975: score += 35
    elif construction_year < 2000: score += 28
    elif construction_year < 2018: score += 14
    soil_risk = {"ZD": 14, "ZE": 20, "ZF": 25}.get(soil_type, 8)
    score += soil_risk
    if visible_damage: score += 20
    ...
    return {"score": min(score, 100), "level": level, ...}
```

4 MCP Client Implementation

4.1 Streamable HTTP Transport

The client in `seismic_client.py` communicates with the MCP server over **streamable HTTP** using the official `mcp-python` SDK. This transport runs JSON-RPC 2.0 messages over a persistent HTTP connection:

```
from mcp.client.streamable_http import streamable_http_client
```

```
class _mcp_session:
    async def __aenter__(self) -> ClientSession:
        self._http_client = httpx.AsyncClient(
            timeout=httpx.Timeout(12.0, read=60.0)
        )
        self._transport = streamable_http_client(
            MCP_SERVER_URL, http_client=self._http_client
        )
        read, write, _ = await self._transport.__aenter__()
        self._session = ClientSession(read, write)
        return await self._session.__aenter__()
```

4.2 Graph Workflow Integration: `call_mcp_tool`

Any LangGraph node calls earthquake data through MCP via:

```
async def call_mcp_tool(name: str, arguments: dict) -> dict:
    async with _mcp_session() as session:
        await session.initialize()
        result = await session.call_tool(name, arguments)
    return {
        "tool": name,
        "result": _extract_tool_payload(result),
        "error": None,
    }
```

The `initialize()` call performs the MCP handshake and negotiates the protocol version before any tool is invoked.

4.3 Demo Walkthrough: `run_mcp_demo`

For the live demo page, `run_mcp_demo` performs all protocol steps and returns them as a structured log that the Angular UI renders step-by-step:

```
async def run_mcp_demo(tool_name, arguments) -> dict:
    async with _mcp_session() as session:
        init   = await session.initialize()      # step 1
        listed = await session.list_tools()      # step 2
        result = await session.call_tool(        # step 3
                     tool_name, arguments)

    return {
        "transport": "streamable-http",
        "server":    { "name": init.serverInfo.name, ... },
        "steps":     [ connect, initialize, list_tools,
                      call_tool, structured_result ],
        "tools":     [_tool_to_dict(t) for t in listed.tools],
        "result":    _extract_tool_payload(result),
        "explanation": [...],
    }
```

5 LangGraph Integration

5.1 Hybrid Classifier and Tool Routing

The chat graph classifies each user question using a keyword fast-path followed by an LLM structured-output fallback, then routes to the corresponding MCP tool. Figure 2 illustrates the topology.

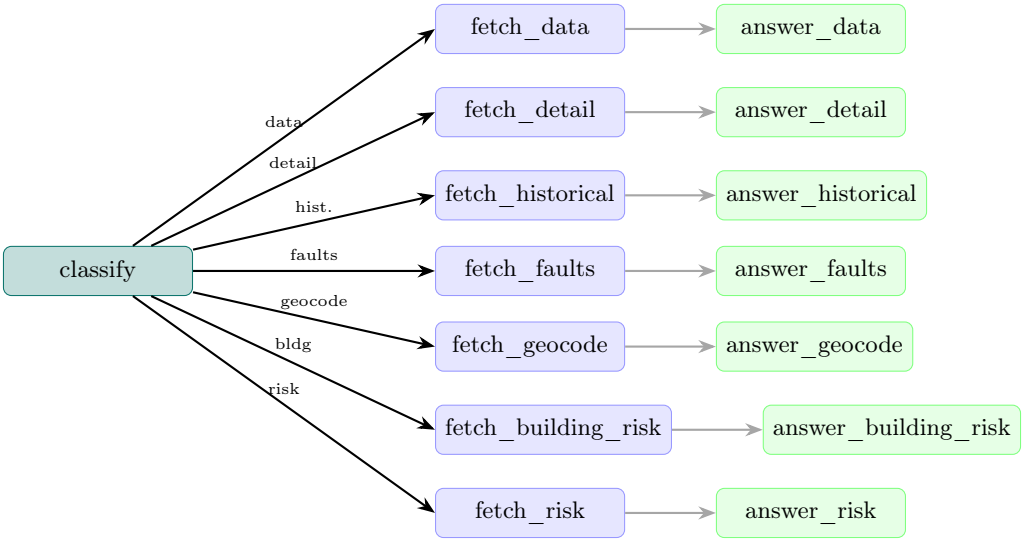

Figure 2: Chat graph topology. Each category routes to the appropriate MCP fetch node, then to an LLM answer node.

5.2 Category-to-Tool Mapping

Category	MCP Tool	Example trigger
`data`	`get_recent_earthquakes`	"Bu hafta hangi depremler oldu?"
`detail`	`get_earthquake_detail`	"us7000m9g4 depremi hakkında bilgi"
`historical`	`get_historical_events`	"Tarihsel büyük depremler"
`gaps`	`get_seismic_gaps`	"Sismik boşluk analizi"
`faults`	`get_fault_lines`	"Aktif fay hatları nerede?"
`geocode`	`geocode_address`	"İzmir Konak koordinatı"
`building_risk`	`assess_building_risk`	"Binam depreme dayanıklı mı?"
`risk`	`get_seismic_context`	"Bölgem için risk nedir?"
`guide`	— (LLM only)	"Deprem çantasında ne olmalı?"
`smalltalk`	— (LLM only)	"Merhaba"

Table 3: Classifier categories and their MCP tool assignments.

5.3 Error Handling

Every fetch node in the graph checks the `error` field returned by `call_mcp_tool`. If the MCP server is unreachable, the answer node immediately returns a user-friendly Turkish message instead of raising an exception:

```
def _mcp_error_response(mcp_call: dict) -> str | None:
    error = mcp_call.get("error")
    if not error:
```

```
        return None
    return (
        "Uzgunum, su anda veri servisine ulasilamiyor. "
        "Lutfen birkac dakika sonra tekrar deneyin. "
        f"(Teknik detay: {error})"
    )
```

6 Live Demo Page

6.1 Architecture

The MCP Demo page (`/mcp-demo`) in the Angular frontend allows any of the five representative tools to be selected, parameterised, and invoked in real time. The request passes through:

$$Angular POST/api/graph/mcp-demo Springproxy forward FastAPI/graph/mcp-demo \texttt{run_mcp_demo()} M$$

6.2 Demonstration Tools

Five tools were selected for the demo page because they each show a distinct capability and produce results that are immediately readable:

Demo Tool	Parameters	What it shows
Son Depremler	hours, min_magnitude, limit	Real-time data retrieval
Sismik Bagflam	latitude, longitude, radius_km	Multi-source parallel fetch
Deprem Detayi	event_id	Event enrichment pipeline
Tarihsel Olaylar	years, min_magnitude	Historical archive query
Bina Risk Degerlendirmesi	lat, lon, year, floors, soil...	Composite scoring

Table 4: Tools exposed on the MCP Demo page.

6.3 Protocol Steps Rendered in the UI

For every demo run, the Angular page renders the five protocol steps returned by `run_mcp_demo`:

1. **MCP HTTP bagflantisi kuruldu** — client connects to `http://localhost:8002/mcp/`
2. **MCP initialize** — protocol version negotiated, server name and capabilities exchanged
3. **tools/list** — all 8 tool schemas discovered at runtime
4. **tools/call** *<tool_name>* — selected tool invoked with typed arguments
5. **Yapilandirilmis sonuc** — JSON result returned to Angular

This step-by-step rendering makes the protocol transparent and directly demonstrates the *discover then invoke* pattern that is central to MCP.

7 Design Decisions

7.1 Streamable HTTP vs. stdio Transport

MCP supports two transports: **stdio** (subprocess) and **streamable HTTP** (persistent HTTP connection). The initial design used stdio, but streamable HTTP was chosen for the following reasons:

Design Rationale

- **No subprocess overhead:** The server is mounted directly into FastAPI; no extra process is spawned per request.
- **External accessibility:** Any external MCP host (Claude Desktop, Cursor) can connect over HTTP without needing local filesystem access.
- **Lifecycle management:** FastMCP's `session_manager` handles connection pooling and cleanup inside the FastAPI lifespan context.

7.2 Single MCP Server for Multiple Consumers

Rather than writing bespoke data-fetching code in the chat graph, the building-risk graph, and the demo page separately, all three consumers call the same MCP server. This means:

- Tool logic is implemented *once* and tested *once*.
- Changing a Spring endpoint URL requires updating only `seismic_server.py`, not every graph.
- Adding a new tool automatically makes it available to the chat agent on the next deploy, without touching the graph code.

7.3 Deterministic Tool vs. LLM Node

`assess_building_risk` returns a deterministic score (rule engine, no LLM). The `/graph/building-risk` endpoint, by contrast, runs the full LangGraph agentic loop with LLM narrative generation and an evaluator feedback loop. The two coexist because:

Key Insight

The MCP tool is fast (no LLM tokens consumed), consistent (same inputs always produce the same score), and suitable for the chat agent where the user only needs a quick number. The full graph endpoint is used when the user explicitly navigates to the Building Risk page and needs a detailed Turkish narrative.

8 Test Scenarios

The following scenarios were used to validate the integration during class demonstration. Each scenario was executed via the MCP Demo page.

8.1 Scenario 1: Real-Time Earthquake Query

Tool: `get_recent_earthquakes`
Arguments: `hours=24, min_magnitude=2.0, limit=8`

Result: PASS

The MCP server fetched live data from Spring, returned a compact record list with `count`, `strongest`, `averageMagnitude` and `summary` fields. The Angular demo page rendered the earthquake list with magnitude badges.

8.2 Scenario 2: Multi-Source Seismic Context

Tool: `get_seismic_context`
Arguments: `latitude=41.015, longitude=28.979, radius_km=100`

Result: PASS

Three Spring endpoints were called in parallel via `asyncio.gather` (recent, historical, fault-lines). The response included `recentCount7Days`, `historicalCount50Years`, `faultFeatureCount`, and `sampleFaultNames` — demonstrating the multi-source aggregation capability.

8.3 Scenario 3: Building Risk Assessment

Tool: `assess_building_risk`
Arguments: `year=1985, floors=7, soil=ZD, visible_damage=false`

Result: PASS

Score of 62/100 ("yuksek" risk) was returned. The `drivers` list correctly identified the pre-2000 construction year and ZD soil classification as the primary contributors. No LLM call was made.

8.4 Scenario 4: AI Assistant — MCP in the Chat Flow

User message: "Son 48 saatte M3.0 ve üzeri deprem oldu mu?"

Result: PASS

The hybrid classifier assigned category `data` via the keyword "son ... deprem". The `fetch_data` node called `get_recent_earthquakes` with extracted arguments `hours=48, min_magnitude=3.0`. The LLM `answer_data` node used the MCP result as its sole data source to generate a natural Turkish response.

9 Reflection and Conclusion

Reflection

MCP removes integration glue. Before MCP, each graph in LangGraph had its own `SpringClient` calls scattered across nodes. After MCP, all data access is centralised in eight well-documented tools. Adding a new data capability now means writing one `@mcp.tool()` function, after which every consumer (chat agent, demo page, external host) gets it automatically.

Reflection

Tool discovery is the key differentiator. Unlike a plain REST API, MCP exposes *structured schemas* at runtime. Claude Desktop or Cursor can read these schemas and decide on their own which tool to call for a given user query — without any custom prompt engineering from the developer.

Reflection

Same server, different consumers. The fact that the LangGraph chat agent and the Angular demo page both use the same `/mcp/` endpoint — alongside the theoretical ability of Claude Desktop to do the same — is the strongest practical argument for the protocol. One server serves many clients.

Reflection

Deterministic tools reduce LLM cost. `assess_building_risk` produces a consistent, auditable score with zero LLM tokens. The full LangGraph agentic loop is reserved for when a narrative analysis is genuinely needed. MCP made this separation clean: the tool is fast and cheap; the graph is thorough and expensive.

Reflection

Error handling must be explicit. MCP does not prevent the server from going down. Every `call_mcp_tool` result must be inspected for the `error` field, and the graph must respond gracefully. Treating MCP as a reliable local function call leads to unhandled exceptions in production.

In conclusion, Model Context Protocol provided a clean, standards-based layer between the Seismic Command data infrastructure and all AI consumers of that data. The integration reduced code duplication, enabled external AI hosts to connect without additional development, and made the AI assistant's data access auditable and observable through the live demo page.